The Default Pre-Processing Pipeline (DPPP) [1]

DPPP (default pre-processing pipeline) is the initial processing routine for LOFAR data. It is capable of flagging, averaging, gain calibration, and various other operations described below (see following sections for more details). It is possible to define an arbitrary operation in Python (or C++), which could be used for trying out new operations. The list of functions that can be carried out using DPPP are

- Flagging (automatic or manual) including the AOFlagger
- Average in time and/or frequency
- Phase shift to another phase center
- Count flags and writing the counts into a table for plotting purposes.
- Combine subbands into a single MeasurementSet
- Demix and subtract A-team sources outside the field of view
- Add stations to form a superstation (for long baselines)
- Filter out stations, baselines, channels, times
- Predict a sky model
- Apply or de-apply the LOFAR beam model
- Do gain calibration
- Apply calibration solutions
- An arbitrary step defined in user code (C++ or Python)

Each of these operations works on the output of the previous one in a streaming way (thus without intermediate I/O to disk). The operations can be combined in any way. The same type of operation can be used multiple times with probably different parameters. The operations are defined in a so-called **parset** file, a text file containing key-value pairs defining the operations (steps) and their parameters.

The ability to execute an arbitrary DPPP step is implemented by means of dynamically loadable shared libraries or by a script written in Python. Especially the latter is a nice way to add experimental operations to DPPP. It is described in more on the LOFAR wiki.

The input to DPPP is any (regularly shaped) MeasurementSet (MS). Regularly shaped means that all time slots in the MS must contain the same baselines and channels. Furthermore, the MS should contain only one spectral window; for a multiband MS this can be achieved by specifying which spectral window to use. The data in the given column are piped through the steps defined in the **parset** file and finally written. This makes it possible to e.g. flag at the full resolution,

average, flag on a lower resolution scale, perform more averaging, and write the averaged data to a new MeasurementSet. If time slots are missing, flagged data containing zeroes are inserted to make the data nicely contiguous for later processing.

The output can be a new MeasurementSet, but it is also possible to update the flags or data in the input. When doing operations changing the meta data (e.g., averaging or phase-shifting), it is not possible to update the input MeasurementSet. Any step can be followed by an output step creating (or updating) a MeasurementSet, whereafter the data can be processed further, possibly creating another MeasurementSet.

It is possible to combine multiple MeasurementSets into a single spectral window. In this way multiple subbands can be combined into a single MeasurementSet. Note this is different from python-casacore's **msconcat** command, because **msconcat** keeps the individual spectral windows, while DPPP combines them into one.

Detailed information on DPPP can be found here. For specific questions regarding this software, you can contact the software developers, Tammo Jan Dijkema or Ger van Diepen.

How to run DPPP

The proper environment has to be defined to be able to run LOFAR programs such as DPPP. Normally this is done by the command:

```
module load lofar
```

Some parameters have to be given to DPPP telling the operations to perform. Usually they are defined in a **parset** file (see section on the parset file for details). The DPPP task can be run like:

```
DPPP some.parset
```

It is also possible to specify parset keys on the command line, which will be added to the keys given in the (optional) parset file or override them. For example:

```
DPPP msin=in.MS steps=[] msout=out.MS
```

does not use a parset file and specifies all keys on the command line. Note this run will copy the given MeasurementSet, while flagging NaN values. The command:

```
DPPP some.parset msout.overwrite=true
```

uses a parset, but adds (or overrides) the key **msout.overwrite**. If no arguments are given, DPPP will try to use the **DPPP.parset** or **NDPPP.parset** file. If not found, DPPP will print some help info.

An example **NDPPP.parset** file can be found in the LOFAR GitHub Cookbook repository: https://github.com/lofar-astron/LOFAR-Cookbook/tree/master/Parset

DPPP will print outputs to the screen or a logger, including some brief statistics like the percentage of data being flagged for each antenna and frequency channel.

The next sections give some example DPPP runs. They only show a subset of the available parameters. DPPP is described in more detail on the LOFAR operations wiki which also contains several examples. It also contains a description of all its parameters.

Copy a MeasurementSet and calculate weights

The raw LOFAR MeasurementSets are written in a special way. To make them appear as ordinary MeasurementSets a special so-called storage manager has been developed, the **LofarStMan**. This storage manager is part of the standard LOFAR software, but cannot be used (yet) in a package like CASA. To be able to use CASA to inspect the raw LOFAR data, a copy of the MeasurementSet has to be made which can be done with a parset like:

```
msin = in.ms
msin.autoweight = true
msout = out.ms
steps = []
```

Apart from copying the input MS, it will also flag NaNs and infinite data values. Furthermore the second line means that proper data weights are calculated using the auto-correlations. The latter is very useful, because the LOFAR online system only calculates simple weights from the number of samples used in a data value.

Count flags

The percentages of flagged data per baseline, station, and frequency channel can be made visible using:

```
msin = in.ms
msout =
steps = [count]
```

The output parameter is empty meaning that no output will be written. The **steps** parameter defines the operations to be done which in this case is only counting the flags.

Preprocess a raw LOFAR MS

The following example is much more elaborate and shows how a typical LOFAR MS can be preprocessed:

```
msin = in.ms
msin.startchan = nchan/32    #1
msin.nchan = nchan*30/32
msin.autoweight = true
msout = out.ms
steps = [flag,avg]           #2
flag.type = aoflagger        #3
flag.memoryperc = 25
avg.type = average           #4
avg.freqstep = 60
avg.timestep = 5
```

1. Usually the first and last channels of a raw LOFAR dataset are bad and excluded. Because the number of frequency channels of a LOFAR observations can vary (typical 64 or 256), an elaborate way is used to specify the first and number of channels to use. They are specified as expressions where the 'variable' **nchan** is predefined as the number of input channels. Note that it might be better to flag the channels instead of removing them. In that way no gaps are introduced when concatenating neighbouring subbands.
2. Two operations are done: flagging followed by averaging. The parameters for these steps are specified thereafter using the step names defined in the **steps** parameter. Note that arbitrary step names can be used, but for clarity they should be meaningful.
3. A step is defined by various parameters. Their names have to be prefixed with the step name to connect them to the step. In this way it is also possible to have multiple steps of the same type in a single DPPP run. Usually the type of step has to be defined, unless a step name is used representing a type. In this case the aoflagger is used, an advanced data flagger developed by Andr'e Offringa. This flagger works best for large time windows, so it tries to collect as many data in memory as possible. However, to avoid memory problems this step will not use more than 25% of the available memory.
4. The next step is averaging the data, 60 channels and 5 time slots to a single data point. Averaging is done in a weighted way. The new weight is the sum of the original weights.

In DPPP the data flows from one step to another. In this example the flow is read-flag-average-write. The data of a time slot flows to the next step as soon as a step has processed it. In this case the flag step will buffer a lot of time slots, so it will take a while before the average step receives data. Using its parset parameters, each step decides how many data it needs to buffer.

Update flags using the preflagger

The preflagger step in DPPP makes it possible to flag arbitrary data, for example baselines with international stations:

```
msin = in.ms
msout =
steps = [flag]
flag.type = preflagger
flag.baseline = ![CR]S*
```

No output MS is given meaning that the input MS will be updated. Note that the {tt msout} always needs to be given, so one explicitly needs to tell that an update should be done. The preflagger makes it possible to flag data on various criteria. This example tells that baselines containing a non core or remote station have to be flagged. Note that in this way the baselines are flagged only, not removed. The **Filter** step described hereafter can be used to remove baselines or channels.

Remove baselines and/or channels

The filter step in DPPP makes it possible to remove baselines and/or leading or trailing channels. In fact, it should be phrased in a better way: to keep baselines and channels. An output MS name must be given, because data are removed, thus the meta data changes.

msin = in.ms msout = out.ms steps = [filter] filter.type = filter filter.baseline = [CR]S*&

If this would be the only step, it has the same effect as using **msselect** with **deep=true**. The filter step might be useful to remove, for example, the superterp stations after they have been summed to a single superstation using a **stationadder** step.

The filter step has the option (using the **remove** parameter) to remove the stations not being used from the ANTENNA subtable (and other subtables) and to renumber the remaining stations. This will also remove stations filtered out before, even if done in another program like **msselect**). In this way it can be used to ‘normalize’ a MeasurementSet.

Combining stations into a superstation

The stationadder step in DPPP makes it possible to add stations incoherently forming a superstation. This is particularly useful to combine all superterp stations, but can, for instance, also be used to add up all core stations. This step does not solve or correct for possible phase errors, so that should have been done previously using BBS. However, this might be added to a future version of DPPP.

```
msin = in.ms
msout = out.ms
steps = [add]
add.type = stationadder
add.stations = {CSNew:[CS00[2-6]*]}
```

This example adds stations CS002 till CS006 to form a new station CSNew. The example shows that the parameter **stations** needs to be given in a Python dict-like way. The stations to be added can be given as a vector of glob patterns. In this case only one pattern is given. Note that the wildcard asterix is needed, because the station name ends with LBA or HBA (or even HBA0 or HBA1).

In the example above the autocorrelations of the new station are not written. That can be done by setting parameter **autocorr**. By default they are calculated by summing the autocorrelations of the input stations. By setting parameter **sumauto** to false, they are calculated from the crosscorrelations of the input stations.

Update flags for NaNs

Currently it is possible that BBS writes NaNs in the CORRECTED_DATA column. Such data can easily be flagged by DPPP.

```
msin = in.ms
msin.datacolumn = CORRECTED_DATA
msout = .
steps = []
```

DPPP will always test the input data column for NaNs. However, if no steps are specified, DPPP will not update the flags in the MS. An update can be forced by defining the output name as a dot. Giving the output name the same name as the input has the same effect.

Creating another data column

When updating a MeasurementSet, it is possible to specify another data column. This can, for instance, be used to clone the data column.

```
msin = in.ms
msin.datacolumn = DATA
msout = .
msout.datacolumn = CORRECTED_DATA
steps = []
```

In this way the MeasurementSet will get a new column CORRECTED_DATA containing a copy of DATA. It can be useful when thereafter, for example, a python script operates on CORRECTED_DATA.

Demixing

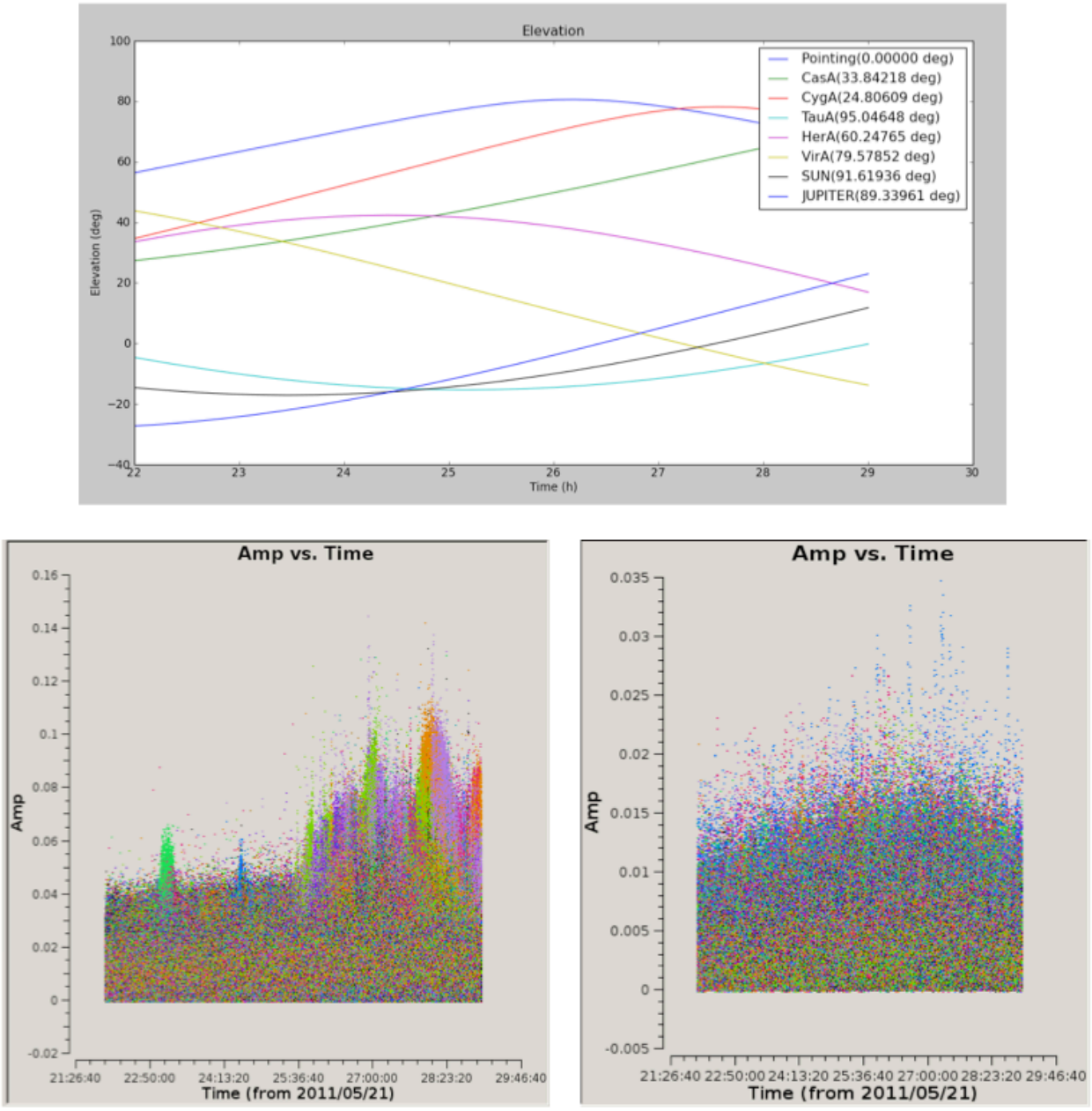

Fig. 5 ***Top:*** *the elevation and angular distance of the A-team from the target field centered on B1835+62. This plot shows that during this observation CygA and CasA are very high in elevation and pretty close to our target (24 and 33 degrees, respectively).* ***Bottom left:*** *target visibilities before demixing - the interference of Cas A and Cyg A with the target visibilities is the cause of the bump in the data in the second part of the observation.* ***Bottom right:*** *the contributions of the two A-team sources is gone. This is particularly evident in the second part of the observation.*

The so-called “demixing” procedure should be applied to all LBA (and sometimes HBA) data sets to remove from the target visibilities the interference of the strongest radio sources in the sky (the so called A-team: CasA, CygA, VirA, etc...). Removing this contribution is essential to make it possible to properly calibrate the target field. To understand whether demixing is needed for your data, you are suggested to inspect the elevation of the A-team sources during your observation. By combining this information with the angular distance of the A-team from your

target, you can have a clear picture of how critical is to apply this algorithm to your data to improve the calibration and imaging of the visibilities. This overview is provided by the script **plot_Ateam_elevation.py**, which is described in the chapter on data inspection.

There are two ways to do the demixing:

- The old demixer will demix in the same way for the entire observation without taking temporal variations into account. One can define:
    - The baselines to use.
    - The (A-team) sources to solve for and to subtract.
    - If the target has to be ignored, solved or deprojected.
    - Possibly different time and frequency averaging factors to use for demix and subtract.
- Recently a new (smart)demixing scheme (designed by Reinout van Weeren) has been added to DPPP. Basically it works the same as the old demixer, but for each time window it estimates the data by evaluating a rough model of the A-team sources and target. Using those data it tests which sources have to be demixed, which baselines should be used, and if the target has to be ignored, solved, or deprojected. The LOFAR beam is taken into account in estimate, solve, and subtract. In this scheme one can specify:
    - A detailed model of the A-team sources to be used in the solve/subtract.
    - A rough model of the A-team sources to be used in the estimation. If not given, the detailed model is used.
    - A model of the target field which can be obtained using e.g. gsm.py.
    - The baselines to be used in the demixer. Note that the estimation step might exclude baselines for a given time window.
    - Various threshold and ratio values to test which sources, etc. to use.

Below, an example old demixer parset is given

```
msin = in.ms
msout = out.ms
steps = [demix]
demix.type = demixer
demix.subtractsources=[CygA, CasA, VirA]
demix.targetsource=3C196
demix.freqstep=16
demix.timestep=10
```

Following this example, the source models of CygA, CasA, and VirA will be subtracted with the gain solutions calculated for them. The target source model is also used to get better gain solutions for the A-team sources.

If no source model is given for the target, the target direction is projected away when calculating the gains. This should not be done if an A-team source is close to the target. Currently, Science Support is investigating how close it can be. If too close, one should specify

```
demix.ignoretarget=true
```

Examples of demixing performance on real data are given in Fig. 5 shown above.

Combine measurement sets

For further processing it can be useful to combine preprocessed and calibrated LOFAR MeasurementSets for various subbands into a single MeasurementSet. In this way BBS can run faster and can a single image be created from the combined subbands.

```
msin = somedirectory/L23456_SAP000_SB*_uv.MS.dppp
msin.datacolumn = CORRECTED_DATA
msin.baseline = [CR]S*&
msout = L23456_SAP000_SBcomb_uv.MS.dppp
steps = []
```

The first line shows that a wildcarded MS name can be given, so all MeasurementSets with a name matching the pattern will be used. The data of all subbands are combined into a single subband and the meta frequency info will be updated accordingly. The second line means that the data in the CORRECTED_DATA column will be used and written as the DATA column in the output MS. The third line means that only the cross-correlations of the core and remote stations are selected and written into the output MS. Note this is different from flagging the baselines as shown in the preflagger example. Input selection means that non-matching baselines are fully omitted, while the preflagger only flags baselines. Note that no further operations are needed, thus no steps are given. However, it is perfectly possible to include any other step. In this case one could use **count**.

It is important to note that subbands to be combined should be consecutive, thus contiguous in frequency. Otherwise BBS might not be able to handle the MS. This means that the first and last channels of an MS should not be removed, but flagged instead using the preflagger.

The ParSet File

As shown in the examples in the previous section, the steps to perform the flagging and/or averaging of the data have to be defined in the parset file. The steps are executed in the given order, where the data are piped from one step to the other until all data are processed. Each

step has a name to be used thereafter as a prefix in the keyword names specifying the type and parameters of the step.

A description of all the parameters that can be used in DPPP can be found on the LOFAR wiki.

Input parameters (msin)

The **msin** step defines which MS and which DATA column to use. It is possible to skip leading or trailing channels. It sets flags for invalid data (NaN or infinite). Dummy, fully flagged data with correct UVW coordinates will be inserted for missing time slots in the MS. Missing time slots at the beginning or end of the MS can be detected by giving the correct start and end time. This is particularly useful for the imaging pipeline where BBS requires that the MSs of all sub bands of an observation have the same time slots. When updating an MS, those inserted slots are temporary and not put back into the MS.

When combining multiple MSs into a single one, the names of the input MSs can be given in two ways using the **msin** argument.

- The name can be wildcarded as done in, say, bash using the characters *, ?, [], and/or {}. The directory part of the name cannot be wildcarded though. For example,

```
msin=L23456_SAP000_SB*_uv.MS
```

- A list of MS names can be given like

```
**msin=[in1.ms, in2.ms]**
```

The MSs will be ordered in frequency unless **msin.orderms=false** is given.

It is possible to select baselines to use from the input MS. If a selection is given, all baselines not selected will be omitted from the output. Note this is different from the Preflagger where data flags can be set, but always keeps the baselines.

LOFAR data are written and processed on the CEP2 cluster in Groningen. This cluster consists of the head node lhn001 and the compute/storage nodes locus001..100. Different subbands are stored on different nodes, and it may be necessary to search them all for the required data. MeasurementSets are named in the format **LXXXXX_SAPnnn_SBmmm_uv.MS**, where **L** stands for LOFAR, **XXXXX** is the observation number, **nnn** is the subarray pointing (beam), and mmm is the subband. An example measurement set name is **L32667_SAP000_SB010_uv.MS**.

Output parameters (msout)

The **msout** step defines the output. The input MS is updated if an emtpy output name is given.

If you are working on CEP3, note that data should be written to **/data/scratch/<username>** (which may need creating initially). Output data should **not** be written back to the storage disks. Also, do not write output data to **/home/<user name>**, as space is very limited on this disk.

You can let **DPPP** create a so-called VDS file, which tells other data processing programs (notably, **BBS** and **mwimager**) where the data live. You need a so-called cluster description file for this. These can be found in the LOFAR Cookbook GitHub repository. For the curious, the cluster description is a simple ASCII file that should be straightforward to understand).

Flagging

The properties of the flagging performed through DPPP can be summarized as follows.

- If one correlation is flagged, all correlations will be flagged.
- The msin step flags data containing NaNs or infinite numbers.
- A **PreFlagger** step can be used to flag (or unflag) on time, baseline, elevation, azimuth, simple uv-distance, channel, frequency, amplitude, phase, real, and imaginary. Multiple values (or ranges) can be given for one or more of those keywords. A keyword matches if the data matches one of the values. The results of all given keywords are AND-ed. For example, only data matching given channels and baselines are flagged. Keywords can be grouped in a set making it a single (super) keyword. Such sets can be OR-ed or AND-ed. It makes it possible to flag, for example, channel 1-4 for baseline A and channel 34-36 for baseline B. Below it is explained in a bit more detail.
- A **UVWFlagger** step can be used to flag on UVW coordinates in meters and/or wavelengths. It is possible to base the UVW coordinates on a given phase center. If no phase center is given, the UVW coordinates in the input MS are used.
- **A MADFlagger step can be used to flag on the amplitudes of the data.**
    - It flags based on the median of the absolute difference of the amplitudes and the median of the amplitudes. It uses a running median with a box of the given size (number of channels and time slots). It is a rather expensive flagging method with usually good results.
    - It is possible to specify which correlations to use in the MADFlagger. Flagging on XX only, can save a factor 4 in performance.
- **An AOFlagger step can be used to flag using the AOFlagger.**
    - Usually it is faster than using **rficonsole** itself, because it does not reorder the data. Instead it flags in a user-defined time window. It is possible to specify a time window overlap to reduce possible edge effects. The larger the time window, the better the

flagging results. It is possible to specify the time window by means of the amount of memory to be used.
    - The flagging strategy can be given in an rficonsole strategy file. Such a file should not contain a 'baseline iteration' command, because DPPP itself iterates over the baselines. Default strategy files exist for LBA and HBA observations (named LBAdefault and HBAdefault).
    - Note that the **AOFlagger** flags more data if there is a large percentage of zero data in the time window. This might happen if zero data is inserted by DPPP for missing time slots in a MeasurementSet or for missing subbands when concatenating the MeasurementSets of multiple subbands.
    - By default the QUALITY subtables containing flagging statistics are written. They can be inspected using **aoqplot**.

Averaging

The properties of the averaging performed through DPPP can be summarized as follows.

- Unflagged visibility data are averaged in frequency and/or time taking the weights into account. New weights are calculated as the sum of the old weights.

Some older LOFAR MSs have weight 0 for unflagged data points. These weights are set to 1.

- The UVW coordinates are also averaged (not recalculated).
- It fills the new column LOFAR_FULL_RES_FLAG with the flags at the original resolution for the channels selected from the input MS. It can be used by BBS to deal with bandwidth and time smearing.
- Averaging in frequency requires that the average factor fits integrally. E.g. one cannot average every 5 channels when having 256 channels.
- When averaging in time, dummy time slots will be inserted for the ones missing at the end. In that way the output MeasurementSet is still regular in time.
- An averaged point can be flagged if too few unflagged input points were available

Flag statistics

Several steps shows statistics during output about flagged data points.

- A flagger step shows the percentage of visibilities flagged by that step. It shows:
    - The percentages per baseline and per station.
    - The percentages per channel.
    - The number of flagged points per correlation, i.e. which correlation triggered the flagging. This may help in determining which correlations to use in the MADFlagger.

- An AOFlagger, PreFlagger, and UVWFlagger step show the percentage of visibilities flagged by that flagging step. It shows percentages per baseline and per channel.
- The **msin** step shows the number of visibilities flagged because they contain a NaN or infinite value. It is shown which correlation triggered the flagging, so usually only the first correlation is really counted.
- A Counter step can be used to count and show the number of flagged visibilities. Such a step can be inserted at any point to show the cumulative number of flagged visibilities. For example, it can be defined as the first and last step to know how many visibilities have been flagged in total by the various steps.

Furthermore the AOFlagger step will by default write some extra QUALITY subtables in the output MeasurementSet containing statistical information about its performance. These quality data can be inspected using the **aoqplot** tool.

Footnotes

[1] : This section is maintained by Ger van Diepen and Tammo Jan Dijkema.